Dual-View Address Mapping for NPU–PIM Memory

Reproducing the PFM address-mapping optimizer on Ramulator 2

Group 13

Yash Gupta	264101064	yash.g@iitg.ac.in
Nikunj Ladumor	264101032	nikunj.ladumor@iitg.ac.in
Vamshi Kumar Kanduri	264101061	vamshi.kanduri@iitg.ac.in
Priyanshu Ganjeer	264101035	priyanshu.ganje@iitg.ac.in

Department of Computer Science and Engineering, IIT Guwahati

Paper: S. Zhao et al., "Rethinking Unified Memory for NPU-PIM Systems: Dual-View Memory for Dynamic Inference of LLM", arXiv:2608.06989

The problem: one matrix, two devices, two wishes

NPU (prefill, large GEMM)

- reads weights as one long sequential stream
- wants consecutive addresses on *different* banks / bank groups / channels
- so the data bus never waits

PIM (decode, GEMV)

- one small compute unit next to each bank
- one command drives *all banks at once*
- wants each unit's weight column *inside its own bank*

A shared memory stores each weight matrix once, so it has **one** physical layout.
PFM: pick one layout per 2 MB page that is good for both, with an integer program.

Assignment scope

- Implement the address-mapping optimizer (Python + PuLP)
- In Ramulator 2.0, add a mapping choice per memory region
  (we use Ramulator 2.1, the current release)
- Compare three layouts: **NPU-friendly**, **PIM-friendly**, **jointly optimized**
- Report bandwidth utilization for NPU sequential reads and PIM matrix-vector access
- Use the LLaMA3-8B weight shapes

Matrix ($d_{in} \times d_{out}$)	Shape	2 MB pages
q_proj, o_proj	4096×4096	16 each
k_proj, v_proj	4096×1024	4 each
gate_proj, up_proj	4096×14336	56 each
down_proj	14336×4096	56 (64 if padded)

Out of scope: GPU kernels, hardware synthesis, larger models.

DRAM background in two facts

1. **Row buffer.** Reading the open row is fast. Another row costs precharge + activate $= 16 + 16$ cycles (DDR4-2400).
2. **Bank groups.** Two reads to the *same* bank group: 6 cycles apart (t_{CCD_L}). To *different* groups: 4 cycles (t_{CCD_S}).
   A 64 B burst uses the bus for 4 cycles $\Rightarrow$ staying in one group caps bandwidth at $4/6 = 66.7\%$.

Address mapping

Which address bit picks the column, bank group, bank or row.
A 2 MB page has $2^{21}/64 = 32768$ lines $\Rightarrow$ **15 free bits** to assign.
DDR4 x8: 7 column, 2 bank-group, 2 bank, 4 row bits.

On DDR4, bank-group bits play the role of the paper's HBM channel bits.

The PIM "unit window"

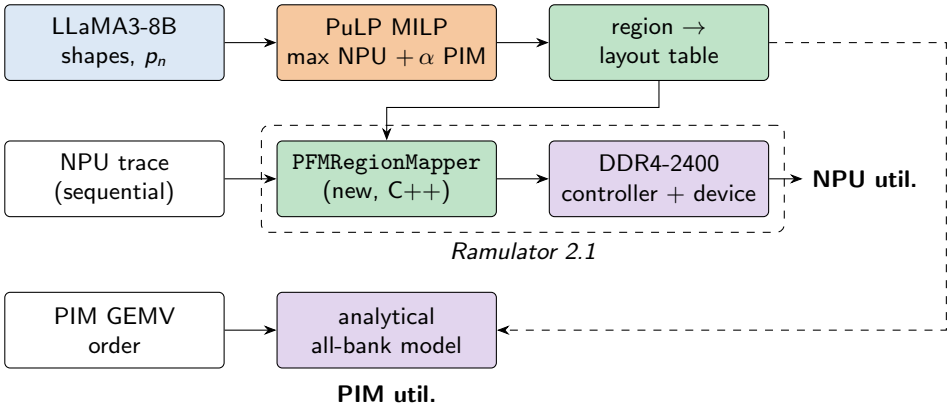
- Weights stored column by column: one column $= d_{in} \times 2\,\mathrm{B} = 2^{p_n}$ lines
  $p_n = 7$ for $d_{in} = 4096$, $p_n = 9$ for $d_{in} = 14336$ (padded to 16384)
- Line index $= c \cdot 2^{p_n} + t$ (column c, offset t)
- 16 PIM units on DDR4 x8 (one per bank); unit u gets columns $c = 16g + u$

$\Rightarrow$ the unit number sits in bits $p_n \ldots p_n{+}3$: the **unit window**

Rule

PIM is fast only if every bit in the unit window is a bank or bank-group bit (then every unit has its own bank).

What we built



- Metric: utilization $= 4 \times$ reads/DRAM cycles, checked against Ramulator's own throughput counter
- Every run checks: layout is one-to-one, no duplicate addresses, all reads reach DRAM

New in Ramulator: a mapping choice per memory region

`PFMRegionMapper`: one new C++ address mapper (about 160 lines)

- splits memory into 2 MB regions
- holds a list of layouts and a table: region → layout
- maps every request with its region's layout
- checks each layout matches the DRAM's bit counts

```
addr_mapper = PFMRegionMapper(
  layouts=["bg1 col6 bank1 col1 bank1 bg1 row4",
           "bg1 col6 row2 col1 bank2 bg1 row2"],
  region_layout=[0,0,0,0,0,0,1,1])
```

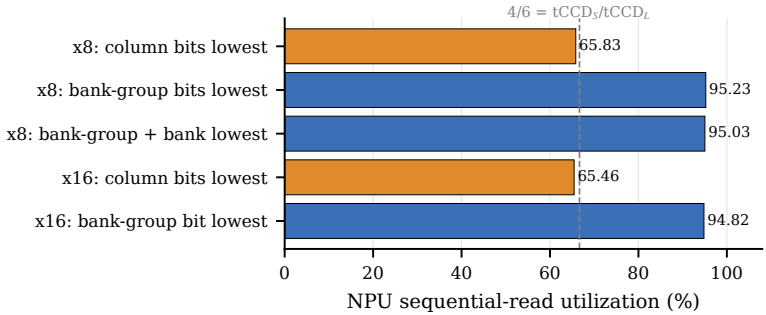
Check 1: all 20 layouts give *identical* cycle counts to an independent Python implementation.

Check 2: one run over 8 regions ($6 \times d_{in}$=4096, $2 \times$ down_proj):

Plan	NPU %
NPU-friendly everywhere	95.05
PIM-friendly everywhere	65.81
PFM α=0.25, 2 layouts	93.70
PFM α=2, 2 layouts	64.70

Switching layouts between regions costs almost nothing.

First measurement: bank-group bits decide NPU bandwidth



- Column bits lowest: every read stays in one bank group ⇒ 66% (the 4/6 limit)
- One bank-group bit at the bottom ⇒ 95%. Row hits are >99% in both cases.

The optimizer (MILP)

Binary $x_{l,j} = 1$ if bit j selects level $l \in \{\mathrm{Co}, \mathrm{BG}, \mathrm{Ba}, \mathrm{Ro}\}$

- Each bit gets exactly one level: $\sum_l x_{l,j} = 1$
- Each level gets exactly its bit count: $\sum_j x_{l,j} = B_l$

NPU score: $\frac{2}{3} + \frac{1}{3}$ [BG bit at position 0] $- \lambda \cdot$ [row bits below column bits]

PIM score: $2^{-(\text{missing bank bits in window})} \cdot 0.5^{(\text{row bits in window})}$

Objective: maximize NPU $+ \alpha \cdot$ PIM (solved with CBC)

Checks

MILP value = exact score of its own answer; it ties or beats 20,000–100,000 random layouts every time; re-running reproduces every stored layout.

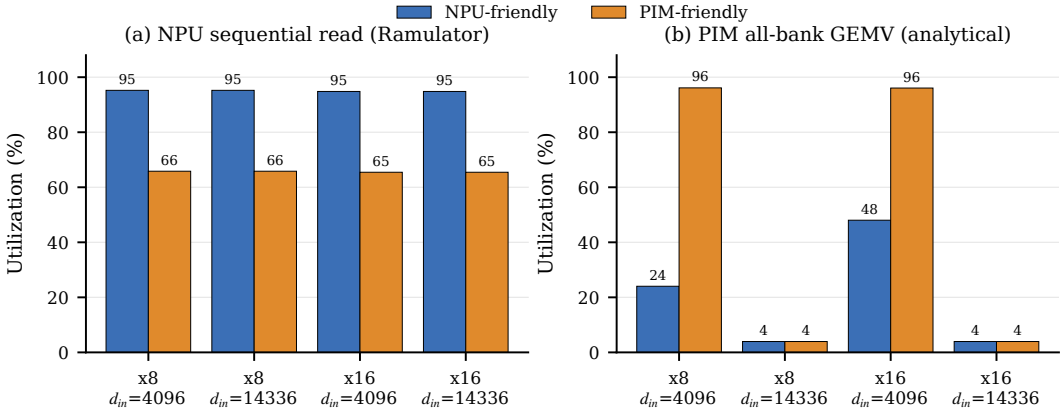
PIM model

Stock Ramulator has no PIM commands, so PIM utilization is computed from Ramulator's DDR4-2400 timings:

- one step = all 16 units read offset t of their current column
- cost per unit: 6 cycles, +16 if the bank is closed, +32 if the row changes
- units in the same bank add up; the step takes as long as the slowest bank
- utilization = ideal time / modeled time

We also replayed the same order as normal reads in Ramulator. The scheduler reorders those reads and hides bank conflicts, so this proxy was too lenient (about 95% for almost every layout). It is kept only as a cross-check.

Baseline results



- Each fixed layout is good for one device only
- For $d_{in} = 14336$ (down_proj) the unit window holds row bits: **4% PIM** for both

Layouts found by the optimizer (DDR4 x8)

d_{in}=4096: **PIM unit window = bits 7..10**

	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NPU-friendly	Ro	Ro	Ro	Ro	Ba	Ba	Co	Co	Co	Co	Co	Co	Co	BG	BG
PIM-friendly	Ro	Ro	Ro	Ro	Ba	Ba	BG	BG	Co	Co	Co	Co	Co	Co	Co
PFM α=0.25	Ro	Ro	Ro	Ro	BG	Ba	Co	Ba	Co	Co	Co	Co	Co	Co	BG
PFM α=1	Ro	Ro	Ro	Ro	BG	Ba	Ba	BG	Co	Co	Co	Co	Co	Co	Co

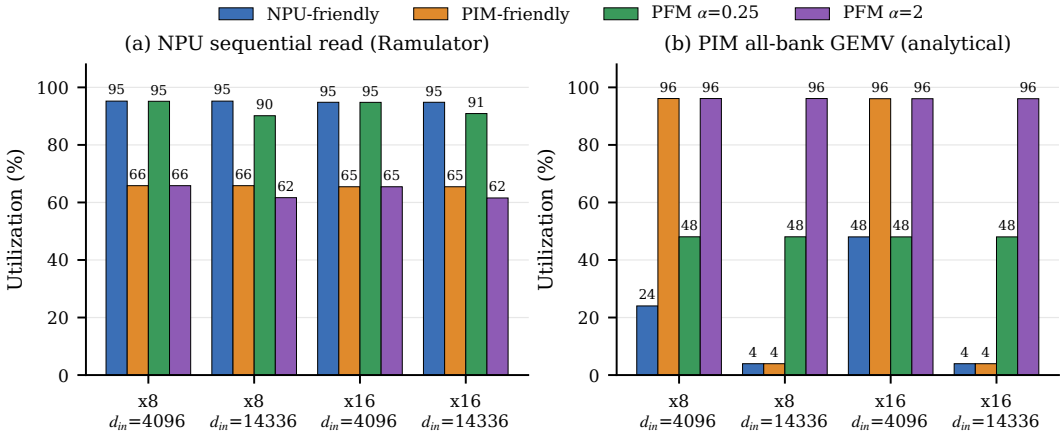
d_{in}=14336: **PIM unit window = bits 9..12**

	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NPU-friendly	Ro	Ro	Ro	Ro	Ba	Ba	Co	Co	Co	Co	Co	Co	Co	BG	BG
PIM-friendly	Ro	Ro	Ro	Ro	Ba	Ba	BG	BG	Co	Co	Co	Co	Co	Co	Co
PFM α=0.25	Ro	Ro	BG	Ba	Ba	Co	Ro	Ro	Co	Co	Co	Co	Co	Co	BG
PFM α=1	Ro	Ro	Ba	BG	BG	Ba	Ro	Ro	Co	Co	Co	Co	Co	Co	Co

line-index bit

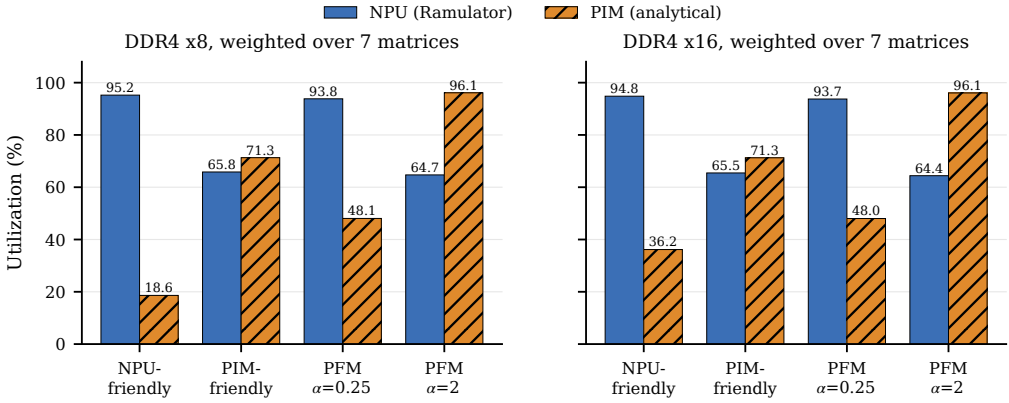
Red box = PIM unit window. Co column, BG bank group, Ba bank, Ro row.

Contribution results



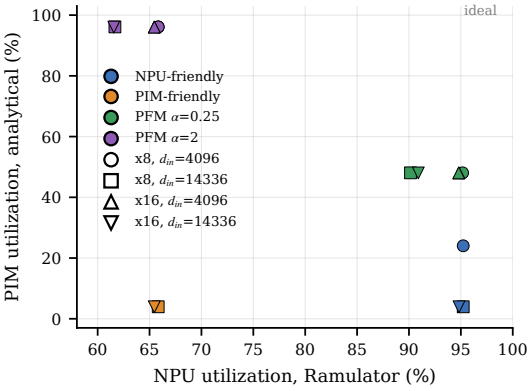
- PFM $\alpha = 0.25$: NPU stays at 90–95%; on x8, PIM goes 24 $\rightarrow$ 48% and 4 $\rightarrow$ 48%
- PFM $\alpha = 2$: best PIM (96%) everywhere, including down_proj
  (note: the fixed baselines were designed for d_{in}=4096; part of the down_proj gain is adapting to its shape)

Whole LLaMA3-8B decoder layer



- $\alpha = 0.25$ vs NPU-friendly: -1.4 NPU points, $+29.4$ PIM points (18.6% $\rightarrow$ 48.1%)
- $\alpha = 2$ vs PIM-friendly: $+24.8$ PIM points (71.3% $\rightarrow$ 96.1%) for -1.1 NPU points

Why no layout wins both on DDR4, and what α does



Bit budget. x8 has only 4 bank-level bits.

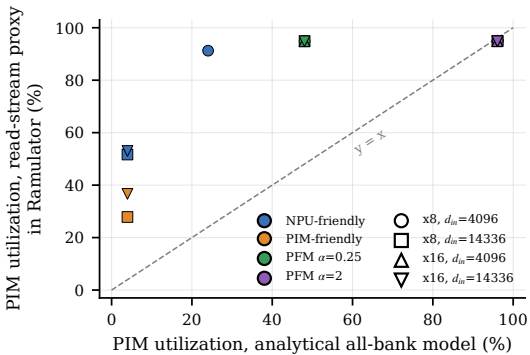
- NPU needs 1 of them at bit 0
- PIM needs all 4 in the unit window

$\Rightarrow$ at most 3 in the window $\Rightarrow$ PIM $\leq$ 50% when NPU is at full speed

The paper's HBM has extra channel bits for the NPU; in our reading, that is why it can serve both.

α picks the point on the trade-off: switch at $\alpha \approx 2/3$.
Low α: prefill-heavy serving. High α: decode-heavy.

How good are the models?



- **NPU score vs Ramulator**: 1.00 ↔ 95.2%, 0.67 ↔ 65.8%, 0.98 ↔ 90.2% (0.98 predicts ≈93%: 3 points optimistic)
- **PIM**: the read proxy says ≈95% for most layouts because the scheduler reorders reads; a lock-step PIM command cannot
- So PIM numbers come from the analytical model

Limitations

1. PIM utilization is analytical, not cycle-simulated; it shares its main assumption with the optimizer
2. Single-channel DDR4, not multi-channel HBM2e-PIM; the paper's numbers are not reproduced
3. The fixed baselines are designed for $d_{in} = 4096$, so they are weak on down_proj
4. Static layouts per tensor class; no runtime scheduling or hardware validation

Conclusion

- Verified PuLP MILP that maps 15 address bits per 2 MB page
- New Ramulator mapper `PFMRegionMapper`: a layout per 2 MB region, checked cycle-for-cycle
- Joint layouts raise PIM utilization at nearly the same NPU utilization: 18.6% $\rightarrow$ 48.1% PIM at 93.8% NPU over a decoder layer
- down_proj: 4% $\rightarrow$ 48–96% PIM
- On single-channel DDR4 the bit budget forces a trade-off, and α selects it

Next: multiple channels, PIM commands inside Ramulator for a cycle-level PIM model.

Source code: `https://github.com/groupiitg/PFM`

Thank you. Questions?